UN RIFERIMENTO BM

API dei giochi bm, volume 1

Grafica, input e sistema: cosa può chiamare una cartuccia Lua, con un primo gioco da seguire.

Indice

Come funziona una cartuccia 4

Le funzioni che il tuo gioco definisce 4

Dimensioni dello schermo 4

Colori e coordinate 5

Limiti 5

Uscire da un gioco 5

Il budget di tempo 5

Il tuo primo gioco 6

Provalo, passo per passo 6

Altri modi per provarlo 6

Copertina 7

Grafica 8

Schermo e forme 8

Sprite 10

La mappa di tile 12

Input 15

Pulsanti 15

Tasti scelti dal giocatore 17

Mouse e puntatore 17

Più giocatori 19

Tempo e sistema 20

`stat(n)` 21

Fotogrammi pesanti: `frameskip` 22

Il dev kit e il profiler 22

Il report dell'ultima sessione 23

Esempi 24

Salvataggi 25

Da sapere 25

Un record 25

Più partite in corso 25

Documenti 27

Da sapere 27

Schermate 28

Per il giocatore 28

Per un'app 28

Nomi e formato 28

Provalo, passo per passo 29

Rete (UDP) 30

Il web 31

Punti aperti 33

LEGGIMI

Che cos'è. Il volume 1 dell'API dei giochi bm: come funziona una cartuccia, un primo gioco passo per passo, poi grafica, input, tempo e sistema, salvataggi, documenti, schermate e rete.

Come usarlo. Fai il primo gioco in ordine, poi tieni le tabelle a portata di mano. Tutto ciò che sta in un riquadro con bordo e ombra va copiato così com'è.

Cosa manca. Ogni lacuna è segnata con un'etichetta evidenziata DA COMPLETARE; non si inventa nulla. Suono, 3D, bmlib, bmnet e gli strumenti sono negli altri volumi.

Come funziona una cartuccia

Una cartuccia `.bm` è un gioco per bm scritto in **Lua 5.4**. Il kernel disegna in C (640×360, colore RGB565 a 16 bit, 60 fotogrammi al secondo); Lua si occupa solo della logica. Esempi completi sono in `carts/` : `pong/` , `snake/` , `shooter/` (solo codice) e `herocart/` (con uno sprite sheet PNG e mappe CSV).

Le funzioni che il tuo gioco definisce

Funzione	Quando
`_init()`	una volta, dopo il caricamento
`_update()`	a ogni fotogramma (60 Hz), prima di `_draw`
`_draw()`	a ogni fotogramma, dopo `_update`
`_exit()`	(facoltativa) Ctrl+Esc, PS o Start+Select: `true` chiude subito; `false` resta (la cartuccia chiede, per esempio "modifiche non salvate: ancora Ctrl+Esc per uscire", e chiude dopo con `quit()` o al secondo Ctrl+Esc)
`_leave()`	(facoltativa) in una partita in rete (`online(true)`), quando il giocatore conferma di uscire: il gioco avvisa il server (o l'host), poi la cartuccia si chiude senza essere sospesa

Metti la logica e l'input in `_update` , e solo il disegno in `_draw` . Lo schermo **non** viene cancellato da solo: `_draw` di solito comincia con `cls()` .

Dimensioni dello schermo

`SCREEN_W` e `SCREEN_H` sono variabili globali che contengono la dimensione dello schermo.

Cartuccia fatta con	`SCREEN_W` × `SCREEN_H`
predefinito	640 × 360
`--res 480x270`	480 × 270
`--res 320x180`	320 × 180
`--res 256x256`	256 × 256
`--res 360x360`	360 × 360 (lo schermo del `.b16`)

La cartuccia può cambiare la risoluzione mentre gira con `screen(w, h)` (vedi Grafica): dal fotogramma successivo valgono le nuove dimensioni.

Colori e coordinate

- I colori sono interi `0xRRGGBB` (per esempio `0xFF8000`, arancione) oppure `rgb(r, g, b)` con valori 0–255. Lo schermo li trasforma in RGB565 (5 bit di rosso, 6 di verde, 5 di blu).
- Le coordinate sono in pixel, (0,0) in alto a sinistra; `w` e `h` sono larghezza e altezza.

Limiti

- Un errore o un ciclo infinito (oltre **20 milioni di istruzioni Lua** in una chiamata di `_init`, `_update` o `_draw`; il contatore riparte a ogni chiamata) ferma la cartuccia senza bloccare la console. L'errore, con la sua riga, appare sulla console.
- Sandbox: niente `io`, `os`, `load`, `dofile`. `require` carica solo le librerie incorporate nel kernel: `"assist"`, `"bm3d"`, `"bmlib"`, `"bmnet"`, `"predict"`, `"words"`, `"padtype"`, `"riff"` e `"bmui"`. Hai `string`, `table`, `math`, `utf8` e `coroutine`.

Uscire da un gioco

- Ctrl+Esc (tastiera), PS o Start+Select (gamepad) tornano al menu di bm lasciando il gioco **sospeso**: resta in memoria, e A sulla sua copertina lo riprende da dove era. Se la cartuccia ha `_exit()`, la interrogano prima.
- Esc da solo, in un gioco, è Start (il suo menu).
- In una **partita in rete** (`online(true)`) il gioco non viene sospeso: il sistema chiede a chi esce, solo sulla sua console, "Leave the match?". Sì chiama `_leave()` e chiude il gioco; indietro resta.
- Dalla linea seriale Ctrl+\ è Ctrl+Esc, e `q` chiude senza chiedere.
- I tasti del sistema e quelli della cartuccia compaiono mentre tieni premuto **F12**.

Il budget di tempo

- 60 fps significa **16,7 ms** per fotogramma per `_update` + `_draw` + la copia sullo schermo. `stat(1)` ti dice quanto ne usa la cartuccia.
- Il disegno è in C: una chiamata a `spr` o `rectfill` costa pochi microsecondi, ma ogni chiamata da Lua ha un costo fisso. Sul Pi: circa 1800 sprite 16×16 chiamati da Lua a 60 fps, circa 4500 da C; circa 1200 triangoli 3D.
- Evita di creare nuove tabelle a ogni fotogramma, se puoi: il garbage collector ha meno lavoro.
- Il testo è in celle da 8×16: per scrivere ben allineato usa x multipli di 8 e y multipli di 16.

Il tuo primo gioco

Farai una palla gialla che muovi con le frecce, su uno schermo blu scuro. Usa solo `cls`, `circfill`, `print` e `btn`.

Provalo, passo per passo

1. Sul tuo computer, nel repository di bm, crea il file `carts/hello/main.lua` con questo codice:

```lua
local x, y = 300, 160

function _init() -- una volta, all'avvio
end

function _update() -- 60 volte al secondo: logica
  if btn(0) then x = x - 3 end -- sinistra
  if btn(1) then x = x + 3 end -- destra
  if btn(2) then y = y - 3 end -- su
  if btn(3) then y = y + 3 end -- giù
end

function _draw() -- 60 volte al secondo: disegno
  cls(0x102040) -- sfondo blu scuro
  circfill(x, y, 20, 0xFFD050) -- una palla gialla
  print("hello from bm!", 8, 8, 0xFFFFFF)
end
```

2. Impacchettalo in una cartuccia. Dovresti ottenere un file `hello.bm` nella cartella corrente.

```sh
python3 scripts/mkbm.py -o hello.bm --lua carts/hello/main.lua --title "Hello" --author "me"
```

3. Mettilo sulla console. Copia `hello.bm` nella cartella `carts/` della scheda SD e accendi. Se la console è già accesa, premi `R` nel menu: la scheda viene riletta.
4. Nel menu trova **Hello** di "me" e avvialo. Dovresti vedere uno schermo blu scuro, una palla gialla e il testo bianco `hello from bm!` in alto a sinistra.
5. Premi le frecce (o WASD): la palla si sposta di 3 pixel a ogni fotogramma.
6. Premi Ctrl+Esc (tastiera) o Start+Select (gamepad) per tornare al menu. Il gioco resta sospeso; A sulla sua copertina lo riprende.

Se invece della palla vedi un errore, il codice Lua è andato storto: il gioco si ferma e il messaggio con il numero di riga appare sulla console. Correggi quella riga e impacchetta di nuovo.

[DA COMPLETARE: shots/api-core-hello.png]

PROVA

Fai cambiare colore alla palla quando premi A. In `_update` aggiungi `if btnp(4) then c = rgb(math.random(255), math.random(255), math.random(255)) end`, dichiara `local c = 0xFFD050` in cima, e disegna la palla con `circfill(x, y, 20, c)`.

Altri modi per provarlo

La build del repository può aggiungere il tuo gioco da sola, e puoi provarlo anche senza un Pi.

- Nella build: metti il gioco in `carts/<name>/main.lua`, aggiungi `<name>` a `GAMES` nel `Makefile` e una riga `title_<name> := Title`. `make` crea `build/carts/<name>.bm`.
- In QEMU, senza un Pi:

DA COPIARE

```
qemu-system-arm -M raspi0 -bios build/kernel.img -serial tcp:127.0.0.1:4444,server=on,wait=off -serial n
python3 tools/bm_load.py tcp:127.0.0.1:4444 --cart hello.bm
```

- Dalla linea seriale (un Pi con un adattatore USB-seriale):

```
DA COPIARE
python3 tools/bm_load.py /dev/ttyUSB0 --cart hello.bm
```

Copertina

`--cover cover.png` è facoltativo: un PNG di qualsiasi dimensione che il menu stampa sulla scheda del gioco, un quadrato di 88×88. Un'immagine quadrata viene rimpicciolita; le altre restano intere, larghe quanto il quadrato, sopra una copia sfocata e più scura di se stesse che riempie il resto. Nella build basta un file `carts/hello/cover.png`. Senza copertina il menu stampa il titolo.

Grafica

Tutto ciò che sta sullo schermo: forme, testo, sprite e mappa di tile.

Schermo e forme

Funzione	Cosa fa
`rgb(r, g, b)`	l'intero colore `0xRRGGBB` di tre numeri 0-255 (i colori dell'API sono interi a 24 bit: `0xFFFFFF` è bianco, un numero piccolo come 7 è quasi nero)
`cls([c])`	riempie lo schermo (nero se manca `c`). Con il 3D sulla GPU l'ARM non lo riempie: il job della GPU pulisce la pagina a quel colore (gratis anche a 1080p); se il 2D viene prima del 3D l'ARM riempie come sempre
`screen(w, h)`	imposta la risoluzione della cartuccia dal fotogramma successivo: `true` , oppure `false` se non è una delle dimensioni qui sotto
`screen()`	restituisce larghezza e altezza di adesso
`screen(i)`	restituisce la i-esima dimensione della lista (da 1), oppure `nil` . Su RGB30 (schermo quadrato) la lista è 360×360, 720×720 e 720×405 (16:9, centrata, con barre nere)
`screen_modes()`	restituisce le dimensioni che `screen()` accetta su questo dispositivo come tabella `{ {w, h}, ... }` (nell'ordine di `screen(i)`), e l'indice di quella attuale (`nil` se la dimensione non è nessuna)
`function _screen(w, h)`	un hook: se la definisci, il runtime la chiama una volta prima di `_init` con la prima dimensione, e all'inizio di ogni fotogramma che segue un cambio di dimensione. Disponi l'HUD qui
`safe_area()`	restituisce `x, y, w, h, dw, dh` : la parte dello schermo che ha le proporzioni della dimensione nell'header della cartuccia, al centro (un gioco 360×360 su 640×360 dà `140, 0, 360, 360`). Disponi l'HUD lì dentro. `dw, dh` sono la dimensione dell'header
`aspect()`	restituisce larghezza divisa per altezza dello schermo, e `"wide"` , `"square"` oppure `"tall"`
`pset(x, y, c)`	scrive un pixel
`pget(x, y)`	legge un pixel: `0xRRGGBB` , oppure `nil` fuori dallo schermo
`line(x0, y0, x1, y1, c)`	una linea
`rect(x, y, w, h, c)`	un rettangolo vuoto
`rectfill(x, y, w, h, c)`	un rettangolo pieno
`circ(x, y, r, c)`	un cerchio vuoto

Funzione	Cosa fa
`circfill(x, y, r, c)`	un cerchio pieno
`tri(x0, y0, x1, y1, x2, y2, c, [c1, c2])`	un triangolo pieno; con tre colori (uno per angolo) il colore sfuma da un angolo all'altro (Gouraud, con dithering)
`print(text, x, y, [c, scale])`	testo con il font 8×16 (bianco se manca `c`), `scale` volte più grande (1–8: 2 = caratteri 16×32); restituisce la x dopo l'ultimo carattere
`font([name])`	imposta il font di `print` da ora in poi: `"8x16"` (quello normale), `"8x14"` o `"6x12"` (106 colonne per 30 righe a 640×360: per strumenti con molto testo); restituisce larghezza e altezza di un carattere del font attuale
`prompt(name, x, y, [small, scale, player, menu])`	disegna un tasto come un gettone colorato, con l'angolo in alto a sinistra in (x, y); restituisce la x dopo il gettone (dettagli sotto)
`prompt(name, [small, scale, player])`	senza coordinate non disegna: restituisce larghezza e altezza
`camera([x, y])`	sposta tutto il disegno di (–x, –y); senza argomenti lo azzera
`clip([x, y, w, h])`	limita il disegno al rettangolo; senza argomenti tutto lo schermo

Dimensioni per `screen(w, h)`

La lista è 16:9: 320×180, 384×216, 480×270, 640×360, 960×540, 1280×720, 1920×1080. Su una TV 1080p tutte sono a pixel interi tranne 1280×720.

- `SCREEN_W` e `SCREEN_H` cambiano quando lo fai; lo z-buffer, la camera 3D e i buffer di luce seguono, il font e `camera()` restano, `clip()` torna a tutto lo schermo.
- Se la console non può dare la dimensione, resta quella di prima.
- Sull'**RGB30** (un pannello quadrato) la lista è invece le due dimensioni del `.b16`: 360×360 (mostrata grande il doppio) e 720×720, entrambe a pixel interi.
- Una cartuccia quadrata (256×256, 360×360) cambia come le altre: sul Pi allora riempie la TV, e la GPU può disegnarvi il 3D (nel riquadro al centro lo disegna l'ARM).
- Il rasterizzatore dell'ARM paga ogni pixel: sul Pi sopra 640×360 serve la GPU.

Gettoni di pulsanti e tasti: `prompt`

`prompt` disegna un tasto come un gettone colorato (il set delle app, `src/kernel/prompts.c`). È alto 16 pixel accanto al testo 8×16, 12 con `small` (da solo quando il font è `"6x12"`), `scale` volte più grande (1–8, come `print`).

`name`	Cosa viene disegnato
maiuscolo: `"A"`, `"B"`, `"X"`, `"Y"`, `"START"`, `"SELECT"`, `"L1"` … `"R3"`, `"UP"`, `"UPDOWN"`, `"LEFTRIGHT"`, `"DPAD"`, `"LSTICK"`, `"PS"`, `"TOUCHPAD"`	i pulsanti del pad, disegnati come sul pad usato per ultimo: un DS4 (croce, cerchio, quadrato, triangolo, OPTIONS, SHARE) finché non se ne usa un altro, che li ha con le lettere

`name`	Cosa viene disegnato
`"RGB30_A"`, `"RGB30_B"`, `"RGB30_X"`, `"RGB30_Y"`	i pulsanti propri dell'RGB30, su qualsiasi console
minuscolo: `"mouse"`, `"mouse_l"`, `"mouse_r"`	il mouse; `"mouse_l"` e `"mouse_r"` con il pulsante sinistro o destro in rosso
minuscolo: `"enter"`, `"esc"`, `"tab"`, `"space"`, `"up"`, `"f1"` ... oppure un carattere (`"s"`, `"1"`, `"+"`)	i tasti della tastiera, con i nomi di `keyp()`
`"ok"`, `"back"` e le azioni di `keymap()`	il loro pulsante (sull'RGB30 `"ok"` è B e `"back"` A, scambiati da `confirm=a`: il gettone è il pulsante premuto davvero)

- Sull'**RGB30** `"A"`, `"B"`, `"X"`, `"Y"` sono i pulsanti propri della console (disposizione Nintendo: X in alto, A a destra, B in basso, Y a sinistra), un pulsante scuro con la lettera del suo colore: A verde, B blu, X rosso, Y giallo (finché non si usa un pad Bluetooth).
- Con `player` (1–4, dopo `scale`: `prompt(name, x, y, small, scale, player)`) il pulsante è disegnato come lo mostra il controller di quel giocatore: il simbolo del DS4, la lettera di un pad o il tasto della tastiera che lo preme (`"space"` per A).
- Con `menu` true (il settimo argomento) ottieni invece il set del menu: tasti bianchi, rialzati su un bordo, sempre alti 16 pixel (`small` viene ignorato).

```
local x = prompt("ok", 16, 200)            -- il gettone; restituisce la x dopo di esso
print("start", x + 4, 200, 0xFFFFFF)
prompt("RGB30_X", 16, 220)                 -- la X rossa dell'RGB30, su qualsiasi console
```

[DA COMPLETARE: shots/api-core-prompt.png]

Sprite

Lo **sprite sheet** è un'immagine divisa in celle 8×8: la cella `n` sta alla colonna `n % (width/8)` e alla riga `n // (width/8)`.

- Viene da `--sheet sheet.png` (PNG RGB o RGBA; alpha sotto 128 = trasparente). Senza PNG è uno sheet trasparente 256×256 da disegnare con `sset` (32 celle per riga: vedi `carts/shooter`).
- Lo sheet può arrivare a 4096×4096 pixel.
- Uno sheet grande con al massimo 256 colori entra nella cartuccia con `--sheet sheet.png --sheet8`: una tavolozza e sequenze ripetute (RLE) invece di 4 byte per pixel, decodificate al caricamento (Titan Clash: 2048×3376 pixel in 1,7 MB).

Funzione	Cosa fa
`spr(n, x, y, [w, h, flip_x, flip_y])`	disegna la cella `n` (w×h celle, predefinito 1×1), anche specchiata
`sspr(sx, sy, sw, sh, dx, dy, [flip_x, flip_y, zoom])`	copia un rettangolo qualsiasi dello sheet; con `zoom` (predefinito 1) lo disegna più grande (`2`, `3` ...) o più piccolo (`0.5`), pixel per pixel: copre `sw * zoom` × `sh * zoom` pixel

Funzione	Cosa fa
`stack(sx, sy, w, h, n, x, y, [rz, scale, step])`	sprite stacking: disegna `n` fette di `w × h` pixel, una accanto all'altra sullo sheet da `(sx, sy)`, da sinistra a destra. La fetta 1 è la più bassa. `(x, y)` è il centro della fetta più bassa. Ogni fetta ruota di `rz` radianti attorno al suo centro, è `scale` volte più grande e sta `step * scale` pixel sopra quella sotto (`step` 1 di norma). Il colore 0 è trasparente
`stackv(sx, sy, w, h, n, x, y, [rz, scale, step])`	come `stack`, ma le fette stanno una sotto l'altra sullo sheet, dall'alto in basso; quella in alto è la fetta 1
`zstack(name, x, y, [rz, scale])`	disegna una zona stack del file delle zone (la riga `stack name x y w h n`, o `stackv` per fette una sotto l'altra)
`sget(x, y)`	legge un pixel dello sheet (`nil` = trasparente)
`sset(x, y, [c])`	scrive un pixel dello sheet (`nil` = trasparente)

```
spr(0, x, y)                  -- cella 0 (8x8)
spr(2, x, y, 2, 2)            -- 2x2 celle dalla cella 2: uno sprite 16x16
spr(2, x, y, 2, 2, true)      -- specchiato in orizzontale
sspr(sx, sy, w, h, x, y)      -- un rettangolo qualsiasi dello sheet, in pixel
```

Zone con nome

La sezione SPRITES dà un nome a rettangoli dello sheet, con i loro fotogrammi (le caselle della stessa misura a destra della prima) e la velocità dell'animazione. Il codice non ha più bisogno delle coordinate.

Funzione	Cosa fa
`zspr(name, x, y, [frame, flip_x, flip_y, zoom])`	disegna una **zona con nome** dello sheet: `frame` da 1 al numero di fotogrammi (oltre ricomincia), oppure `nil`: quello che i suoi fps danno a `time()`, così l'animazione va da sola. Restituisce il fotogramma disegnato. Una zona che non c'è è un errore
`zone(name)`	restituisce `x, y, w, h, frames, fps, slices` della zona (il suo primo fotogramma, in pixel dello sheet; `slices` è 0 per una zona normale, da 2 a 255 per una zona stack), oppure `nil`
`zones()`	i nomi delle zone, in ordine (`{}` se non ce ne sono)
`zboxes(name, [frame, kind])`	gli **hitbox e hurtbox** dei fotogrammi di una zona (vedi sotto)

```
zspr("coin", x, y)                    -- l'animazione della moneta, va da sola
zspr("hero_run", x, y, f, to_the_left)
local _, _, w, h = zone("hero_run")   -- la sua misura, per le collisioni
```

Hitbox e hurtbox dei fotogrammi

Sotto una zona, nel file delle zone, scrivi le sue caselle: `hurt` dove il personaggio può essere colpito, `hit` dove il suo colpo fa male, `body` lo spazio che occupa (perché i corpi non si attraversino), con il

fotogramma (da 1, oppure `*` per ogni fotogramma) e il rettangolo dall'angolo del fotogramma.

`zboxes` restituisce una lista di `{x, y, w, h, kind, frame}` :

- `x` e `y` sono dall'angolo in alto a sinistra del fotogramma (la casella può sporgere).
- `kind` è `"hurt"` (dove può essere colpito), `"hit"` (dove fa male), `"body"` (lo spazio che il corpo occupa) o un numero del gioco (3–255).
- `frame` è 0 per una casella di ogni fotogramma.
- Con `frame` (da 1; oltre l'ultimo ricomincia) ottieni le sue caselle e quelle di ogni fotogramma; senza, tutte quelle della zona.
- `kind` (un nome o un numero) tiene un solo tipo.
- Ottieni `{}` se non ce ne sono; una zona che non c'è è un errore.

Il gioco le mette nel mondo (anche specchiate) con `lib.hits` di bmlib (vedi il volume su bmlib).

```
# name  x  y  w  h  frames fps
punch   0  64 32 32 4 12
  hurt  *  8  2 16 30        # il corpo, in ogni fotogramma
  hit   3  24 10 10  6       # il pugno, solo nel fotogramma 3
```

```
for _, b in ipairs(zboxes("punch", f, "hit")) do
  rect(x + b.x, y + b.y, b.w, b.h, 0xFF4040)     -- per vederle mentre fai il gioco
end
```

La mappa di tile

La mappa viene da `--map map.csv` : una riga di numeri separati da virgole per ogni riga della mappa, ogni numero una cella dello sheet (0 = vuota). Senza mappa sono 256×256 celle vuote, un solo livello.

Funzione	Cosa fa
`map(mx, my, [x, y, mw, mh, layer, mask])`	disegna la mappa dalla cella (mx, my), mw×mh celle, in (x, y). `layer` : il numero (da 1) o il nome del livello (predefinito 1). `mask` : solo le celle il cui tile ha almeno uno di quei **flag** (`fget` ; 0 o mancante: tutte)
`mget(mx, my, [layer])`	legge una cella della mappa (0 = vuota; fuori dalla mappa dà 0)
`mset(mx, my, n, [layer])`	scrive una cella della mappa
`msize([w, h])`	restituisce `w, h, layers` : la misura della mappa in celle e quanti livelli ha; con `w e h` ogni livello prende quella misura, le celle che ci stanno restano dove erano
`mlayers([list])`	restituisce i nomi dei livelli, in ordine (il primo è disegnato sul fondo). Con una lista (da 1 a 8 voci) la mappa prende quei livelli: una voce è un nome (il livello con quel nome, o uno nuovo vuoto) oppure `{name, from}` (una copia del livello `from` , un numero o un nome): così si aggiungono, spostano, rinominano e tolgono
`fget(n, [f])`	i **flag** del tile (cella dello sheet) `n` : un byte, 8 flag; con `f` (0–7) quel flag, `true o false`
`fset(n, f, on)`	accende o spegne il flag `f` del tile `n`

Funzione	Cosa fa
`fset(n, byte)`	con due argomenti scrive tutti gli 8 flag insieme
`mflags(x, y, [w, h, layer])`	i flag dei tile che il rettangolo tocca, **in pixel** (la mappa disegnata in 0, 0; senza `w` e `h` un punto), messi insieme (or); 0 = niente. La cella 0 e l'esterno della mappa non ne hanno. Per le collisioni: `mflags(x, y + h, w, 1) & 1 ~= 0` (flag 0 sotto i piedi)

```
camera(cam_x, cam_y)                                  -- scorrimento
map(cam_x // 8, cam_y // 8, cam_x // 8 * 8, cam_y // 8 * 8, 81, 46)   -- solo la parte che si vede
local t = mget(px // 8, py // 8)                      -- la cella sotto un punto
mset(cx, cy, 5)                                       -- cambiare una cella (porte, oggetti presi)
```

Livelli

La mappa ha da 1 a 8 **livelli** della stessa misura, ciascuno con un nome (il primo si chiama `"main"` se non gliene dai un altro). Vengono disegnati uno alla volta, nell'ordine che vuoi: per esempio lo sfondo, poi gli sprite, poi il livello che va **davanti** al personaggio (cime degli alberi, archi, tetti). Ogni livello si legge e si scrive per conto suo.

```
cls(0)
map(0, 0, 0, 0, 80, 45, "back")     -- il pavimento
spr(hero, x, y)
map(0, 0, 0, 0, 80, 45, "front")    -- le cime degli alberi sopra l'eroe
mset(cx, cy, 0, "front")            -- ogni livello si legge e si scrive per conto suo
```

Flag: cos'è ogni tile

Ogni tile dello sheet ha 8 flag (0–7) che dicono al gioco cos'è: muro, acqua, scala, pericolo… Le collisioni guardano i flag invece dei numeri dei tile, quindi puoi aggiungere tile nuovi senza toccare il codice. La convenzione di bmlib (ogni gioco può sceglierne un'altra):

Flag	Valore	Significato
0	1	solido: muri, pavimenti
1	2	piattaforma: si attraversa da sotto, ci si sta sopra da sopra
2	4	scala
3	8	acqua
4	16	fa male (spuntoni, lava)
5–7		liberi per il gioco

```
fset(12, 0, true)                          -- anche il tile 12 è solido
if fget(mget(cx, cy), 0) then ... end      -- la cella è solida
if mflags(px, py + 8, 8, 1) & 1 ~= 0 then  -- il personaggio (8x8) sta su qualcosa
  on_ground = true
end
map(0, 0, 0, 0, 80, 45, 1, 4)             -- solo le scale
```

Da dove vengono sprite, mappe e flag

Con `mkbm.py`:

- `--map map.csv` è il primo livello; ogni altro `--map name=file.csv` è un livello in più (fino a 8, nell'ordine dato; un CSV più piccolo del primo viene riempito con celle vuote).
- `--flags flags.csv` dà i flag: numeri 0–255 per le celle 0, 1, 2… in ordine, oppure `n=flags` per la cella `n`.
- `--sprites zones.txt` dà le zone, una per riga: `name x y w h [frames [fps]]`. Sotto una zona, le sue caselle: `hurt`, `hit` o `body frame x y w h`, oppure `box kind frame x y w h` per un tipo del gioco. I nomi `hurt`, `hit`, `body` e `box` non sono zone.
- Nella build del repository bastano i file `map_<name>.csv` (in ordine con `layers_<game> := name ...` nel `Makefile`), `flags.csv` e `sprites.txt` nella cartella del gioco.

```
# flags.csv: il tile 1 è un muro (1), i tile 2 e 3 piattaforme (2), il tile 9 una scala (4)
1=1 2 2
9=4
```

Sulla console:

- Nella pagina della mappa dell'SDK `l` passa al livello successivo, Shift+L ne aggiunge uno (anche dal menu, *New map layer*), `o` mostra solo quello, `c` i flag dei tile.
- Nella pagina degli sprite i tasti `0` – `7` accendono e spengono (alternano) i flag della cella.
- bm Pixel e `scripts/bmres.py` fanno le zone (`bmres.py` porta le caselle con le loro zone).
- `cart_save` e `cart_load` portano livelli, flag, zone e caselle con la cartuccia.
- Il formato è nelle sezioni LAYERS (12), FLAGS (13), SPRITES (11) e BOXES (14) di `src/bm/bm.h`. Un kernel precedente disegna il primo livello e ignora il resto.

Input

Pulsanti, tastiera, mouse e giocatori.

Pulsanti

`btn(i)` è vero finché il pulsante è tenuto premuto, `btnp(i)` solo nel fotogramma in cui viene premuto.

Funzione	Cosa fa
`btn(i, [p])`	`true` finché il pulsante è tenuto premuto; senza `p` da **qualsiasi** controller, con `p` = 1–4 solo da quello del giocatore `p`
`btnp(i, [p])`	`true` solo nel fotogramma in cui viene premuto (lo stesso `p`, anche con un nome)
`keymap(t)`	assegna le **azioni** del gioco ai pulsanti (vedi sotto)
`keymap()`	restituisce la tabella
`keymap(nil)`	toglie la tabella
`controls([scheme])`	la rimappatura del giocatore da Impostazioni > Controls: `controls()` la legge, `controls("scroller"|"topdown"|"stick")` sceglie lo schema del gioco (lo fa `lib.input`)
`controller([p])`	restituisce con cosa gioca il giocatore `p` (1–4, il primo se manca) (vedi sotto)
`online([on, note])`	dice al sistema che la partita si gioca in **rete** (vedi sotto)
`players()`	quanti giocatori hanno un controller (almeno 1) e, come secondo valore, quali: il bit `n` = giocatore `n+1` (per esempio `3, 7` = giocatori 1, 2 e 3)
`stick([p, n])`	lo stick sinistro del giocatore `p`: `x, y` tra –1 e 1 (x verso destra, y verso il basso), con una zona morta; con la tastiera o un pad senza stick è la croce (8 direzioni). Con `n = 1` lo stick **destro** (per mirare negli sparatutto; `0, 0` senza stick). Senza `p`: quello spinto di più

Al posto di `i`, `btn` e `btnp` accettano anche un **nome**:

- un pulsante: `"a"`, `"b"`, `"x"`, `"y"`, `"left"` … `"down"`, `"start"`, `"select"`, `"l1"`, `"r1"`, `"l2"`, `"r2"`, `"l3"`, `"r3"`;
- `"ok"` e `"back"`, il sì e l'indietro del sistema: croce e cerchio sul DS4, A e B su un pad Xbox, Spazio e X sulla tastiera; sull'RGB30 come dicono `confirm=` e `game_buttons=`;
- un'azione di `keymap()`.

`i`	Pulsante	Tastiera	Gamepad	Seriale
0	sinistra	← o A	croce / stick	`a` o ←
1	destra	→ o D	croce / stick	`d` o →

i	Pulsante	Tastiera	Gamepad	Seriale
2	su	↑ o W	croce / stick	`w` o ↑
3	giù	↓ o S	croce / stick	`s` o ↓
4	**A**	spazio, Z, J	A / croce (DS4)	spazio, `j`
5	**B**	X, K	B / cerchio	`x` , `k`
6	**X**	C, L	X / quadrato	`c` , `l`
7	**Y**	V, I	Y / triangolo	`v` , `i`
8	**Start**	Invio	Start / Options	Invio
9	**Select**	Tab	Select / Share	Tab

- Start+Select insieme (o il pulsante PS) chiudono sempre la cartuccia: Start da solo è libero per la pausa del gioco.
- Una cartuccia che non chiede mai `btn(6)` / `btn(7)` (o `btnp`) riceve X come A e Y come B: i giochi a due pulsanti funzionano con tutti e quattro.

Azioni: `keymap`

```
keymap({ jump = "a", fire = {"x", "r1"}, pause = "start", confirm = "ok" })
```

- Fino a 32 azioni, 4 pulsanti ciascuna. Poi usa `btn("jump", p)` , `btnp("fire")` e `prompt("jump", x, y)` .
- Il gioco cambia i tasti richiamando `keymap` (il suo menu di opzioni; li conserva con `save()`).
- Un pulsante sconosciuto, o un'azione con il nome di un pulsante, è un errore.

Con cosa gioca un giocatore: `controller`

`controller` restituisce `{kind, layout, bluetooth, ok, back}` e `color` :

- `kind` : `"keyboard"` , `"ds4"` , `"xbox"` , `"pad"` , `"builtin"` o `"none"` .
- `layout` : `"keyboard"` , `"ds4"` , `"xbox"` , `"nintendo"` o `"none"` . Dice come si chiamano i pulsanti: i simboli del DS4; lettere con A in basso come Xbox; A a destra come l'RGB30.
- `bluetooth` : un booleano.
- `ok` (`"a"` o `"b"`) e `back` (`"b"` o `"a"`): quali pulsanti del gioco sono il sì e l'indietro.
- `color` : il colore del giocatore, quello della luce del suo pad, come intero RGB: `0x3070FF` (giocatore 1, blu), `0xFF3C28` (2, rosso), `0x28D848` (3, verde), `0xFF38A8` (4, rosa).

Partite in rete: `online`

- Con `online(true)` , PS, Ctrl+Esc e Start+Select non sospendono il gioco (gli altri continuano a giocare) ma chiedono a chi esce, solo sulla sua console, "Leave the match?" (uscirà dal gioco e si disconnetterà dal server), sopra il gioco che prosegue.
- Finché la domanda è aperta il gioco non vede né pulsanti né stick né tasti. ok (croce sul DS4, Invio o Spazio sulla tastiera) o di nuovo PS escono: `_leave()` e la cartuccia si chiude. Indietro (cerchio, Esc) resta, e i pulsanti ancora premuti tornano al gioco solo dopo essere stati rilasciati.
- `note` è una riga sotto la domanda (per esempio `"You are the host: the match ends for all."`).

- `online(false)` alla fine della partita. `online()` restituisce se era in rete e se la domanda è aperta (il giocatore è via).

Tasti scelti dal giocatore

Una cartuccia che lascia scegliere i tasti al giocatore (come nano8) legge la tastiera tasto per tasto e i controller pulsante per pulsante.

Funzione	Cosa fa
`rawkeys(on)`	con `true` le tastiere smettono di essere controller per `btn()` e `pad()`: si leggono con `keydown()`. Ctrl+Esc chiude comunque la cartuccia
`keydown(u)`	`true` finché il tasto con uso USB HID `u` è premuto (USB o Bluetooth): `0x04` … `0x1D` le lettere A–Z, `0x1E` … `0x27` le cifre, `0x28` Invio, `0x2C` spazio, `0x4F` … `0x52` le frecce (destra, sinistra, giù, su), `0xE0` … `0xE7` Ctrl, Shift, Alt, GUI a sinistra e poi a destra
`keys()`	gli usi dei tasti premuti ora (`{0x1D, 0xE1}`): per "premi un tasto"
`pad([p])`	i pulsanti che il giocatore `p` (1–4) tiene premuti, come bit: 1 sinistra, 2 destra, 4 su, 8 giù, 16 A, 32 B, 64 Start, 128 Select, 256 X, 512 Y, 1024 L1, 2048 R1, 4096 L2, 8192 R2, 16384 L3, 32768 R3 (i grilletti e gli stick premuti: DS4 e Xbox 360; sui pad generici i pulsanti 7–8 e 11–12); senza `p` quelli di tutti. I tasti seriali contano come il controller del primo giocatore (L1 e R1: `u` e `o` dalla linea seriale, Q ed E dalla tastiera USB; Select: Tab da entrambe)
`lastinput()`	cosa è stato premuto per ultimo: `"keyboard"`, `"ds4"` o `"pad"` (un altro controller); `nil` prima di qualsiasi tasto. Per mostrare i tasti giusti con `prompt()` (per esempio `"enter"` o `"A"`)

Mouse e puntatore

Una cartuccia ha il puntatore solo se lo chiede: senza `mouse(true)` non c'è (nel menu di bm c'è sempre). Lo muove solo un mouse USB o Bluetooth: un controller non lo mostra mai (il suo stick destro è `stick(p, 1)`). La console può averlo spento per tutto il sistema (`mouse=off` in `bm/config.txt`).

Funzione	Cosa fa
`mouse(on, [arrow])`	`mouse(true)`: la cartuccia vuole il puntatore, e la console disegna la sua freccia sopra il fotogramma (`mouse(true, false)`: niente freccia, la cartuccia disegna il suo cursore); `mouse(false)` lo toglie. Restituisce `false` se la console ha il mouse spento

Funzione	Cosa fa
`mouse()`	restituisce `x, y, buttons, wheel, visible` : la posizione in pixel dello schermo della cartuccia (senza `camera`), i pulsanti premuti come bit (1 sinistro, 2 destro, 4 centrale), i click della rotella in questo fotogramma (su positivo) e `true` se il puntatore si vede (appena qualcosa lo muove). `nil` se la cartuccia non lo ha chiesto o se nulla lo muove (né mouse né stick destro)
`mousep([i])`	`true` nel fotogramma in cui il pulsante `i` viene premuto (0 sinistro, il predefinito; 1 destro; 2 centrale)

```
function _init() mouse(true) end
function _update()
  local x, y = mouse()
  if x and mousep() then tx, ty = x, y end   -- un click
end
```

bmui, il mouse degli strumenti di bm

`require "bmui"` (`src/script/bmui.lua`). Lo usano gli editor (SDK, bm Code, bm Studio, bm Animator, bm Mesh, bm Pixel, Sound, il pannello dell'assistente), e lo può usare anche un tuo gioco o strumento. La sua regola: il mouse fa ciò che fanno i tasti. Le cose diventano cliccabili mentre vengono disegnate, e un click preme il tasto dietro di esse o sceglie ciò che mostrano; il pulsante destro apre un menu contestuale le cui voci sono tasti.

Funzione	Cosa fa
`U.update()`	chiamala per prima in `_update` (legge `mouse()` una volta a fotogramma)
`U.begin()`	chiamala per prima in `_draw` (dimentica le zone del fotogramma prima)
`U.zone(x, y, w, h, kind, a, b)`	qualcosa di cliccabile, mentre viene disegnato (vince l'ultimo disegnato: una finestra sopra la pagina)
`U.at()`	la zona sotto il puntatore
`U.click([b])`	la zona premuta in questo fotogramma (0 sinistro, 1 destro, 2 centrale)
`U.clicked(b)`	un pulsante rilasciato senza trascinamento (la zona dove era stato premuto, o `true`)
`U.key_zone(x, y, w, h, keys)`	un gettone (`"ctrl s"` , `{"shift", "l"}` , `"f5"`) che preme il suo tasto quando viene cliccato
`U.keys()`	preme il tasto del gettone cliccato in questo fotogramma
`U.press(key)`	un tasto come se fosse digitato: il prossimo `keyp()` lo dà (bmui mette il proprio `keyp` per primo)

Funzione	Cosa fa
`U.menu(items, [x, y])`	il menu contestuale al puntatore: `{ {"Copy", "^c"}, "-", {"Run", function() ... end} }` (il tasto mostrato a destra, "Ctrl+C"); righe sulla griglia 8x16
`U.menu_update()`	in `_update` (true: ha preso l'input)
`U.menu_draw()`	per ultima in `_draw`
`U.x` , `U.y` , `U.on` , `U.moved` , `U.wheel` , `U.double` , `U.drag`	il puntatore (`U.on` : un mouse lo ha mosso e si vede), se si è mosso, i click della rotella del fotogramma, un doppio click, il trascinamento `{b, x0, y0, dx, dy}` (oltre 3 pixel; `dx, dy` lo spostamento del fotogramma)
`U.pressed(b)` , `U.released(b)` , `U.down(b)` , `U.shift()` , `U.drag_arrows(step)`	gli stati dei pulsanti, il tasto Shift, e un trascinamento come tasti freccia

Negli editor: un click sceglie una riga, un doppio click (o un click su quella scelta nelle finestre) la prende, la rotella sposta o cambia un valore. Nelle viste 3D il pulsante destro trascinato ruota, quello centrale sposta, la rotella fa lo zoom. Sulle tele il pulsante sinistro disegna e quello centrale preleva il colore. In bm Code un click mette il cursore, un trascinamento seleziona, un doppio click prende la parola, un click nei numeri è un breakpoint.

Più giocatori

Il controller Bluetooth *n* è il giocatore *n*: accoppialo dal monitor con `T` , uno alla volta; ognuno prende il primo posto libero e la sua luce il colore del giocatore (1 blu, 2 rosso, 3 verde, 4 rosa).

- La tastiera o il gamepad USB e la linea seriale sono il primo giocatore senza un pad (senza pad Bluetooth: giocatore 1).
- La tastiera Bluetooth è un giocatore a sé, il successivo senza un pad (il primo, se non c'è nulla su USB).
- I giochi a un giocatore usano `btn(i)` senza `p` e funzionano con qualsiasi controller. Un gioco per più giocatori chiede `btn(i, p)` per ciascuno (esempio: `carts/pong` , modo a due giocatori).
- bmlib ha la schermata dove i giocatori entrano (`lib.party`), lo schermo diviso (`lib.split`) e i colori (`lib.PLAYER_COLORS`); li usa il modello *Versus 2D* dell'SDK.

```
local n = players()                      -- quanti controller ci sono
if btn(2, 1) then p1.y = p1.y - 3 end    -- su, giocatore 1
if btn(2, 2) then p2.y = p2.y - 3 end    -- su, giocatore 2
local x, y = stick(1)                    -- lo stick del giocatore 1, da -1 a 1
```

Tempo e sistema

Funzione	Cosa fa
`time()`	secondi dall'avvio della cartuccia (con i decimali)
`stat(n)`	una misura della console (tabella sotto)
`frameskip([n])`	mantiene il tempo del gioco a 60 `_update` al secondo qualunque sia il costo di `_draw` (vedi sotto)
`devkit([mode])`	l'overlay del dev kit: `0` spento, `1` in alto a destra, `2` in basso a destra (`3` , la vecchia pagina delle funzioni, è quello in alto: i vecchi giochi continuano a funzionare); con un modo lo mostra (un tasto proprio del gioco, per esempio Select su un pad: F11 è quello della tastiera), solo in questa esecuzione: ogni gioco parte come dice Settings (spento di default). Restituisce il modo precedente. Per scorrerli: `devkit((devkit() + 1) % 3)`
`profile([on])`	il profiler delle funzioni (vedi sotto)
`devinfo(line, ...)`	fino a 4 righe del gioco nel report di sessione del dev kit ("the game says:", a fine esecuzione: la sua qualità, i suoi attori…), 18 caratteri ciascuna; `devinfo()` nessuna. Richiamala quando cambiano
`battery()`	la batteria della console: la carica `0 – 100` e `true` se è sul caricabatterie (`local pct, plugged = battery()`); `nil` senza batteria (il Pi). Sull'RGB30 la carica viene dalla tensione (un'indicazione: cala sotto sforzo e sale sul caricabatterie); il caricabatterie si vede entro un quarto di secondo
`battery_low()`	`true` finché la batteria è scarica: 20% o meno fuori dal caricabatterie (fino al 23%). Allora il sistema disegna da solo una piccola batteria rossa in alto a destra sopra il fotogramma (a sinistra dell'overlay del dev kit quando è in alto), in ogni gioco e strumento; *Settings* > *Screen* > *Low battery icon* (`battery_icon=0` in `bm/config.txt`) spegne quell'icona, non `battery_low()` . Solo informazione: ogni console ha la sua batteria, quindi nel lockstep (`bmnet`) non cambia mai ciò che il gioco simula; usala per mostrare qualcosa o per salvare
`code_tokens(text)`	i **token** di un pezzo di codice Lua, contati come fanno `stat(11)` e il dev kit dell'SDK (`src/bm/tokens.c`): ogni nome, parola chiave, numero, stringa e operatore è uno; commenti, spazi, `,` `.` `:` `;` `::` , parentesi di chiusura (`)` `]` `}`), `end` e `local` non contano, né il segno meno davanti a un numero (`-1` è un token). Informazione, non un limite: bm non mette un tetto ai token (nemmeno il `.b16`)
`log(...)`	scrive nel log del kernel (linea seriale e console), non sullo schermo del gioco

Funzione	Cosa fa
`report(kind, text)`	un report per chi sviluppa bm: salvato in `bm/reports` sulla scheda SD con kernel, branch, scheda e data (e una prima riga `options` : le opzioni 3D in vigore), poi inviato al repository dei report se ci sono `github_token` e la rete (`src/kernel/reports.h`); al massimo 8 per esecuzione, 256 KB ciascuno; `true` se salvato. `kind` è una stringa obbligatoria, un'etichetta di categoria a testo libero: viene messa in minuscolo con `-` tra gli altri caratteri e finisce nell'intestazione del report e nel nome del file inviato; con `kind == "session"` l'invio sostituisce il file precedente (Overbit usa `"overbit-bench"`). La prima volta che un gioco lo chiama si chiede al giocatore (la risposta resta in `bm/config.txt` , `allow_...`): dopo un no, `false`
`shot([prefix])`	salva il fotogramma in disegno come PNG (vedi il capitolo Schermate)
`quit()`	chiude la cartuccia a fine fotogramma
`timeslice(co, [k])`	la coroutine `co` si ferma da sola dopo circa `k` migliaia di istruzioni Lua in un fotogramma (400 se manca) e `coroutine.resume` restituisce `true` senza valori: un calcolo lungo prosegue nei fotogrammi successivi invece di fermare la cartuccia al limite di istruzioni. `timeslice(nil)` lo toglie (nano8 lo usa per le sue cartucce)

Numeri casuali: `math.random` . Per giochi diversi a ogni avvio, inizializza il generatore quando il giocatore preme un tasto: `math.randomseed(stat(3))` .

`stat(n)`

n	Valore
0	KB usati da Lua
1	ms dell'ultimo fotogramma (`_update + _draw` , con il 3D della GPU)
2	fps
3	numero del fotogramma
4	triangoli 3D
5	pixel 3D (0 con la GPU)
6	ms spesi nel disegno 3D (da `zclear` ; con la GPU la parte dell'ARM)
7	vertici 3D trasformati
8	ms dall'inizio di questo fotogramma (per misurare le fasi)
9	`1` se la GPU disegna il 3D

n	Valore
10	istruzioni Lua dell'ultimo fotogramma (`_update` + `_draw` , in istruzioni, arrotondate alle migliaia)
11	dev kit: token del codice della cartuccia (`code_tokens`)
12	dev kit: il massimo di KB di Lua di questa esecuzione
13	dev kit: KB dei dati della cartuccia in memoria (sprite sheet, mappa, modelli e scheletri, banco dei suoni, lo z-buffer 3D)
14	dev kit: le istruzioni Lua del fotogramma più carico di questa esecuzione
15	dev kit: quanti `_update` sono girati prima di questo `_draw` (1; di più con `frameskip`)
16	dev kit: ms di tempo di job della GPU (bin + render) contati nell'ultimo fotogramma, `0` quando l'ARM disegna il 3D (insieme a `stat(1)` dice se è la GPU o l'ARM a dare il passo)

ATTENZIONE

`stat` è **solo una misura**: cambia da console a console, quindi non usarla mai (né 1, 2, 6, 8, 10) nella logica del gioco, o una partita in lockstep perde il passo.

Fotogrammi pesanti: `frameskip`

- Quando un fotogramma dura più di 1/60 s, fino a `n` `_update` girano prima del `_draw` successivo (i fotogrammi in mezzo non vengono disegnati), così un gioco che si muove di 1/60 s per `_update` non rallenta.
- Oltre `n` il tempo viene lasciato andare (il gioco rallenta invece di non disegnare mai).
- `1` (il predefinito) è un `_update` per fotogramma; il massimo è 8. Restituisce il valore precedente.
- Un pulsante premuto conta una volta in `btnp()` (e `mousep()` , la rotella) per quanti `_update` lo vedano; `btn()` resta premuto.
- Overbit usa `frameskip(4)` (il suo benchmark `1`).

Il dev kit e il profiler

Il **dev kit** è un overlay di prestazioni sopra qualsiasi gioco, **spento di default**. Accendilo con `perf=top` o `perf=bottom` in `bm/config.txt` (la RGB30 non ha F11: lì usa `perf=`), con F11 sulla tastiera (un tasto di sistema, anche negli strumenti), con `p` dalla linea seriale o con `devkit(mode)` dal gioco: una volta in alto a destra, un'altra in basso a destra, un'altra spento. F11, `p` e `devkit()` durano un'esecuzione: ogni gioco parte (e riprende) come dice l'impostazione salvata. È un piccolo blocco:

```
60fps 8.2ms           fotogrammi al secondo; ms di lavoro in un fotogramma (le parti sotto)
cpu [======---]  49%  il fotogramma della CPU nelle sue parti, % di 16,7 ms
gpu [===------]  31%  il tempo occupato della GPU, % di 16,7 ms (solo mentre la GPU disegna il 3D)
```

- Le parti della barra CPU, ciascuna del suo colore, da sinistra a destra: `_update`, `_draw` senza il suo 3D (con la pulizia e la copia della pagina), le chiamate 3D (`stat(6)` senza l'attesa della GPU), l'interrupt **audio**, l'**attesa della GPU**, e il resto libero.
- L'interrupt audio ferma qualunque codice stia girando: la console ne conta il tempo e lo toglie dalla parte in cui è caduto, così le parti si sommano.
- Le % sono sui **16,7 ms di un fotogramma a 60 Hz** (non sul fotogramma: un gioco a 30 fps mostra oltre il 100%), le medie dell'ultimo secondo. Le % e la cornice della barra sono verdi sotto il 50%, gialle fino all'80%, rosse oltre l'80% o quando in quel secondo è stato saltato un fotogramma.
- Sugli schermi grandi è più grande (x2 da 1280 di larghezza, x3 a 1920). Costa qualche timestamp a fotogramma; spento, nulla. L'icona della batteria scarica sta alla sua sinistra in alto.

Il **profiler delle funzioni**: `profile(true)` misura le funzioni che costano di più, in ms a fotogramma, la media dell'ultimo secondo (60 fotogrammi). Spento non costa nulla.

- `profile()` restituisce fino a 32 funzioni, la più costosa per prima: `{name, where, c, self, total, calls}`, e i fotogrammi misurati (0 durante il primo secondo).
- `where` è `"main.lua:120"` o `"[C]"`; `c` è true per le funzioni della console e quelle di Lua scritte in C; `self` e `total` sono ms a fotogramma, il tempo proprio e con ciò che chiama; `calls` sono le chiamate a fotogramma, solo quelle C.
- `profile(true)` lo accende per l'esecuzione (il report di sessione elenca poi le dieci più costose); `profile(false)` lo spegne (e dimentica).
- Una funzione con un `total` grande e un `self` piccolo passa il tempo nelle altre. Le funzioni della console sono misurate esattamente, quelle Lua a pezzi di 1000 istruzioni (abbastanza per vedere dove va il tempo).

```
for _, f in ipairs(profile()) do log(f.name, f.self) end
```

Il report dell'ultima sessione

Quando un gioco finisce il dev kit scrive un file per gioco nei report, `reports/<branch>/session_<game>_<board>.txt`, che l'esecuzione successiva dello stesso gioco **sostituisce**. Dentro: schermo, fotogrammi e fps, il tempo medio e massimo di `_update + _draw`, la memoria, e il tempo **da un'immagine alla successiva** in sei classi (fino a 18 ms, 25, 34, 50, 100, oltre); il fotogramma nelle sue parti come le misura l'overlay; la quota di tempo CPU dell'interrupt audio; la quota dell'attesa della GPU e la % di fotogrammi in cui **la GPU ha dato il passo** (l'ARM l'ha aspettata più di 2 ms); i 12 fotogrammi più lunghi e le loro parti; con la GPU i suoi job, la memoria del binner e la coda dei fotogrammi; con `profile(true)` le dieci funzioni più costose; e le righe `devinfo()` del gioco a fine esecuzione. Parte dal menu quando c'è la rete.

- In `bm/config.txt`: `session_report=0` mai, `=1` sempre; senza la riga, solo con un `github_token`.
- Dal codice usi `stat(1)`, `stat(2)`, `stat(6)`, `stat(10)`, `stat(11)` – `stat(16)`, `devkit()` e `profile()`. Il limite è 20 milioni di istruzioni per chiamata.
- Il **dev kit dell'SDK** (F1 due volte) ha gli stessi numeri per il progetto: token, le funzioni più grandi, la memoria dei dati, il file contro il segno degli 8 MB di un `.b16` (una guida, non un limite) e i numeri dell'ultima prova (F5).
- A fine esecuzione la linea seriale scrive anche la riga `dev kit: Lua peak ... KB, data ... KB, busiest frame ...k instructions, ... frames over 16.7 ms, ... tokens`.

Esempi

```
local pct, plugged = battery()
if battery_low() and not warned then save(G) warned = true end   -- salva in tempo
log("frame ms", stat(1))                                         -- sulla linea seriale, non sullo scher
```

Salvataggi

Funzione	Cosa fa
`save(t, [slot])`	salva la tabella `t` sulla scheda SD, nello slot 1–8 (predefinito 1); `true`, oppure `false` e il motivo (niente scheda SD, scheda piena…)
`saved([slot])`	la tabella salvata l'ultima volta in quello slot (predefinito 1), oppure `nil`
`saves()`	gli slot in uso, `{[slot] = bytes}` (per esempio `{[1] = 40, [3] = 212}`), e quanti sono (8): per una pagina "carica partita"
`delsave([slot])`	svuota lo slot (predefinito 1); `true`, oppure `false` e il motivo (`"nothing saved in slot 3"`)

Da sapere

- Ogni cartuccia ha **8 slot di salvataggio**, ciascuno una tabella fino a 32 KB, in `/bm/save/` sulla scheda SD.
- Lo slot 1 è `xxxxxxxx.sav` (quello di sempre: `save(t)` e `saved()` senza slot), gli altri `xxxxxxxx.s02` … `.S08`.
- Un uso comune: lo slot 1 per impostazioni e record (lo usa anche `lib.store`), gli altri per le partite in corso.
- Una tabella può contenere numeri, stringhe, booleani e altre tabelle (niente funzioni).
- Le opzioni del gioco nel menu (X sulla copertina) mostrano i byte di tutti gli slot, e *Delete the save data* li cancella tutti.

ATTENZIONE

Il nome dei file di salvataggio dipende da titolo e autore: cambiarli riparte da zero. Scrivere sulla scheda SD richiede qualche millisecondo: chiama `save()` in momenti come la fine di una partita, non a ogni fotogramma (`lib.store` e `lib.best` di bmlib scrivono solo quando un valore cambia).

Un record

Il record di Snake:

```
function _init()
  local data = saved()
  if data then best = data.best end
end
-- alla fine della partita
if score > best then best = score; save({ best = best }) end
```

Più partite in corso

Tre partite in corso, con la pagina per sceglierne una (slot 2, 3 e 4):

```
local function slots()                  -- le righe della pagina "carica partita"
  local used, rows = saves(), {}
  for i = 1, 3 do
    local p = used[i + 1] and saved(i + 1)
    rows[i] = p and ("Game " .. i .. ": level " .. p.level) or ("Game " .. i .. ": empty")
  end
  return rows
end
function save_game(i) save({ level = level, hp = hp }, i + 1) end
function load_game(i) local p = saved(i + 1) if p then level, hp = p.level, p.hp end end
```

Documenti

I file del giocatore, condivisi dalle app: `/docs` sulla scheda SD (bm Write vi tiene i suoi `.BMD`, con le esportazioni `.TXT`, `.MD`, `.HTM` e `.PDF`). Un'app che produce file per il giocatore (testi, disegni, esportazioni) li scrive qui.

Funzione	Cosa fa
`doc_list()`	i file in `/docs`: `{ {name = "letter.bmd", size = 1234}, ... }` (vuota se non ce ne sono); `nil` e il motivo se il giocatore non lo ha permesso
`doc_read(name)`	i byte di un documento (una stringa), oppure `nil` e il motivo
`doc_write(name, bytes)`	scrive un documento, nuovo o sostituito (al massimo 4 MB); `true`, oppure `nil` e il motivo
`doc_delete(name)`	cancella un documento; `true`, oppure `nil` e il motivo

Da sapere

- I nomi sono 8.3 senza cartelle (`"letter.bmd"`; il file viene scritto sempre in minuscolo, e una lettura lo trova in qualsiasi caso). La cartella la crea la prima scrittura.
- La prima volta che un'app li usa la console lo chiede al giocatore, come per la rete. La risposta resta in `bm/config.txt` (`allow_...=docs=yes`), e dopo un no ogni chiamata restituisce `nil` e il motivo.

```
local ok, why = doc_write("notes.txt", "shopping:\nbread\nmilk\n")
if not ok then msg = why end
for _, d in ipairs(doc_list() or {}) do print(d.name .. " " .. d.size) end
```

Schermate

Un gioco o un'app salva il fotogramma che sta disegnando come PNG sulla scheda SD, e il giocatore può fare lo stesso con un tasto.

Per il giocatore

Come	Dove
Print Screen sulla tastiera	`bm/shots/` sulla scheda SD
Select tenuto premuto per 3 secondi su un pad (Share su un DS4)	`bm/shots/` sulla scheda SD

Un riquadro dice `Screenshot saved` e il nome del file per due secondi. L'immagine è lo schermo del gioco com'era stato disegnato: gli overlay del dev kit, l'icona della batteria, il puntatore e quel riquadro **non** ci sono.

> **ATTENZIONE**
> I giochi e le app avviati dal menu di bm la prendono; le schermate del menu stesso non ancora.

[DA COMPLETARE: verificare Select tenuto 3 s sul PowKiddy RGB30; non ancora provato sulla console]

Per un'app

Funzione	Cosa fa
`shot([prefix])`	chiede il fotogramma in disegno: viene salvato come PNG in `bm/shots/` quando `_draw` finisce, e restituisce il nome che avrà (`"bm/shots/PONG0007.png"`). Il nome comincia con le prime lettere del titolo del gioco, o di `prefix` (fino a 4 lettere o cifre), poi un numero. `nil` e un messaggio se i numeri sono finiti

Chiamala in qualsiasi punto del fotogramma; chiamarla due volte in un fotogramma dà un solo file. Non c'è nessuna domanda al giocatore (il file va solo in `bm/shots/`).

```
local path = shot()          -- "bm/shots/PONG0007.png"
local path = shot("menu")    -- "bm/shots/MENU0001.png"
```

Nomi e formato

- **Nome**: `<PREFIX><NNNN>.PNG`, 8.3 come vuole il file system della scheda. Il prefisso sono le prime (fino a) 4 lettere o cifre del titolo del gioco (`Pong` → `PONG`, `Star Shooter` → `STAR`; `SHOT` se non ce ne sono) o del `prefix` dato dall'app.
- **Numero**: uno più del più alto di quel prefisso già presente in `bm/shots/`, da 0001 a 9999. Il Pi Zero non ha un orologio, quindi non c'è la data.
- **Formato**: PNG, RGB a 8 bit, della dimensione dello schermo del gioco (640×360, 360×360…). Non si perde nulla: i colori della console sono a 16 bit (RGB565) e vengono estesi a 8 bit. Niente JPG: rovina il

pixel art.

- **Spazio su disco**: uno schermo uniforme occupa pochi KB; il rumore può occupare quanto l'immagine grezza (circa 0,7 MB a 640×360).

Provalo, passo per passo

1. Avvia un gioco qualsiasi dal menu, per esempio **Pong**.
2. Premi **Print Screen** sulla tastiera, oppure tieni premuto **Select** per 3 secondi sul pad.
3. Aspetta il riquadro che dice `Screenshot saved`.
4. Spegni la console e metti la scheda SD nel computer.
5. Apri la cartella `bm/shots/`: l'immagine è lì, per esempio `pong0001.png`.

Rete (UDP)

Per i giochi in rete. Un gioco sta meglio con la libreria bmnet (lobby, messaggi sicuri, lockstep), che usa queste funzioni: vedi il volume su bmnet.

- Pacchetti UDP fino a 1024 byte; ogni cartuccia ha 2 socket, chiusi quando finisce.
- Gli indirizzi sono testo (`"192.168.1.23"`); `"*"` è il broadcast della LAN.
- Serve la rete della console (WiFi o cavo): senza, `udp_open` restituisce `nil` .
- In bmhost gli stessi pacchetti passano per i socket del PC (`BMHOST_NET_ID=k` per più console sullo stesso PC, `--realtime` per giocare a 60 fotogrammi al secondo).

Funzione	Cosa fa
`s, port = udp_open([port])`	un socket sulla porta (0 o niente: una qualsiasi); `nil` e il motivo se non c'è rete o socket libero. La prima volta che un gioco ne apre uno si chiede al giocatore se può usare la rete (la risposta resta in `bm/config.txt`); dopo un no, `nil` e il motivo
`udp_send(s, address, port, data)`	invia una stringa (al massimo 1024 byte); `true` se è partita (UDP: può perdersi)
`data, address, port = udp_recv(s)`	il prossimo pacchetto arrivato, oppure `nil` ; fino a 48 attendono in coda
`udp_close(s)`	chiude il socket
`net_ip()`	l'indirizzo della console, oppure `nil` senza rete
`net_resolve(name)`	l'indirizzo di un nome: `nil` mentre lo cerca (richiamala al fotogramma dopo), `false` se non esiste

```
local s = udp_open(0)
if s then
  udp_send(s, "*", 4000, "hello")
  local data, address, port = udp_recv(s)
end
```

[DA COMPLETARE: un esempio di due console che si trovano con queste funzioni; le fonti descrivono per questo solo bmnet]

Il web

Per una cartuccia che mostra pagine web. Una richiesta gira in background: il gioco continua a disegnare mentre la pagina arriva. Il kernel fa rispettare le regole qui sotto. Una cartuccia non può cambiarle.

- Il giocatore deve consentire la rete, come per `udp_open`. Senza rete `web_get` restituisce `nil, "no network"`.
- Solo indirizzi `https://`, al massimo 1024 caratteri. Il kernel segue al massimo 3 redirect e rifiuta un redirect verso `http://`.
- Il kernel aspetta che sia passato 1 secondo dall'ultima richiesta allo stesso host. Nel frattempo lo stato è `"wait"`.
- Il corpo si ferma a 4 MB. `web_max_kib` in `bm/config.txt` lo cambia (fino a 32768). Una richiesta può chiedere meno con `max`.
- Una sola richiesta gira alla volta. I risultati di al massimo 3 richieste restano in memoria fino a `web_cancel` o alla fine della cartuccia.
- `web_poll` lavora per circa 4 ms. Una nuova connessione TLS può richiedere circa 100 ms sul Pi Zero W, e il fotogramma aspetta. Mostra un messaggio "connecting" prima del primo `web_poll`.
- Dopo una risposta completa il kernel tiene la connessione per la richiesta successiva allo stesso host (keep-alive). Le chiamate non cambiano.
- Con `resume = "NAME.EXT"` una richiesta fallita o annullata conserva la parte ricevuta in `bm/web/name.prt`. Il `web_get` successivo per lo stesso indirizzo con lo stesso nome chiede al server il resto (`Range`). Il server deve mandare `Accept-Ranges` e una dimensione. Il corpo è sempre il file intero: controlla la dimensione o un hash prima di usarlo.
- `file_write` con un nome che finisce in `.bmm` salva un modello in `/bmm/NAME.bmm`, non in `bm/web`. Il kernel controlla prima i byte, come quando carica un file modello (BMRES, tipo 1, una sezione MESH). Un file che non passa è rifiutato e non scrive nulla. Il nome è da 1 a 8 caratteri `a-z 0-9 _ -`. Un modello che c'è già non viene mai sostituito. Il giocatore non riceve nessuna domanda in più.
- In bmhost `web_get` restituisce `nil` e un motivo. `image_load` e `file_write` funzionano.

Funzione	Che cosa fa
`id = web_get(url, [opts])`	avvia una richiesta GET. `opts`: `max` (un tetto del corpo in byte), `timeout` (millisecondi per una lettura, da 1000 a 60000, predefinito 15000), `resume` (un nome 8.3 semplice). Restituisce l'`id`, o `nil` e il motivo
`state, size, length = web_poll(id)`	fa lavorare la richiesta per circa 4 ms. `state` è `"wait"`, `"busy"`, `"done"` o `"error"`. Chiamala una volta a fotogramma
`web_info(id)`	una tabella con `state`, `status` (lo stato HTTP), `type`, `length`, `location`, `url`, `size`, `resumed` ed `error`. Un 404 finisce in `"done"`: guarda `status`
`web_body(id)`	il corpo come stringa, o `nil` e il motivo. Copia i byte nella memoria di Lua: per decodificare un'immagine o scrivere un file, dai l'`id` a `image_load` o `file_write`
`web_cancel(id)`	ferma la richiesta e libera il risultato

Funzione	Che cosa fa
`img = image_load(bytes_or_id, sx, sy, [maxw, maxh])`	decodifica un JPEG baseline, PNG o WebP nello sprite sheet con l'angolo in alto a sinistra in `(sx, sy)`. JPEG progressivi e immagini sopra i 4 megapixel sono rifiutati. Restituisce `{x =, y =, w =, h =}`, o `nil` e il motivo. Disegnala con `sspr`. Se serve, ingrandisci prima lo sheet con `cart_sheet(w, h)`
`file_write(name, bytes_or_id, [async])`	scrive un file in `bm/web` sulla scheda SD. `name` è un nome 8.3 semplice. Mai un file `.bm`, `.bme` o `.b16`. Al massimo 8 MB (`file_write_max_kib`, fino a 32768) e 128 file nella cartella. Con `async = true` i dati vanno in coda, fino a 4 file aspettano, e i pezzi (fino a 32 KB) vanno sulla scheda mentre la cartuccia chiama `web_poll` o `file_state`
`state, info = file_state(name)`	l'ultima scrittura in coda con quel nome: `"queued"`, `"writing"`, `"done"` o `"error"`. Chiamala una volta a fotogramma mentre una scrittura è in coda

```
if btnp(4) and not req then req = web_get("https://example.org/pic.webp") end
if req then
  local state = web_poll(req)
  if state == "done" then
    img = image_load(req, 0, 64, 160, 90)
    web_cancel(req); req = nil
  elseif state == "error" then
    msg = web_info(req).error
    web_cancel(req); req = nil
  end
end
```

Punti aperti

- **Il tuo primo gioco**: immagine da fare: shots/api-core-hello.png
- **Grafica**: immagine da fare: shots/api-core-prompt.png
- **Schermate**: verificare Select tenuto 3 s sul PowKiddy RGB30; non ancora provato sulla console
- **Rete (UDP)**: un esempio di due console che si trovano con queste funzioni; le fonti descrivono per questo solo bmnet